

Aula 3 — Testes de SPA & PWA

"Como testar PWA offline? Como verificar hydration?"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 08/10/2026

Recap da Aula 2 — o que de fato fizemos

- Playwright por dentro: CDP, locators, auto-waiting, flakiness
- `await expect(...)` — `expect` de locator/page é assíncrono (e precisa de matcher)
- Network mocking com `page.route()` — spec 02, testes 1–4
- `storageState` / `auth.setup.ts` : login uma vez, reaproveitado nos specs

Ficou pra hoje: visual regression (spec 03) e os blocos 5–7 do spec 02 (opcionais).

| Dúvida no spec 02? Chama agora. O bot ainda aceita PR até o prazo no Canvas.

Agenda da aula (3h30)

- **90min** — SPA & PWA: SPA vs hydration, Service Worker, manifest, offline (teoria + demo guiada)
- **15min** — Intervalo
- **90min** — Lab hands-on: specs 03 (visual), 04 (SPA), 05 (PWA/offline) + Lighthouse CI
- **15min** — Wrap-up e entrega

Objetivos da aula

- **Testar** SPAs: esperar o app ficar pronto (hydration), navegação sem recarregar, lazy loading
- **Testar** Service Worker lifecycle e cache strategies
- **Validar** PWAs (manifest, install prompt, offline mode)
- **Configurar** Lighthouse CI bloqueando merge
- **Aplicar** performance budgets como teste

SPA × site tradicional — o que muda pro teste

Site tradicional

cada clique → o **servidor** manda uma **página nova** e a tela recarrega inteira.

SPA (Single Page App)

o navegador baixa o **JavaScript uma vez**; os cliques **trocam a tela sem recarregar**.

Consequências pra quem testa SPA:

1. O app **demora um instante pra ficar pronto** depois de abrir → não dá pra clicar de cara
2. Navegar **não recarrega** → o estado do JS continua vivo (o spec 04 prova isso)
3. Telas viram **chunks carregados sob demanda** (*lazy loading*)
4. Abrir uma URL profunda direto (`/movie/603`) também precisa funcionar (*deep link*)

Hydration não é água (calma, é engenharia)

No dicionário, *hidratar* é beber água. Em software, é **encher de vida algo que já existe, mas está "vazio"**. A palavra aparece em vários lugares:

Onde você ouve	O que "hidratar" significa	Exemplo
Front-end com SSR	o JavaScript "acorda" o HTML que já chegou pronto (liga os cliques)	React + Next.js
Estado do app (<i>rehydrate</i>)	restaurar o que estava salvo quando o app reabre	favoritos voltando do IndexedDB
Banco / ORM	transformar linhas do banco em objetos do código	<code>User.fromRow(...)</code>
Cache	pré-carregar um cache com dados	"cache aquecido"

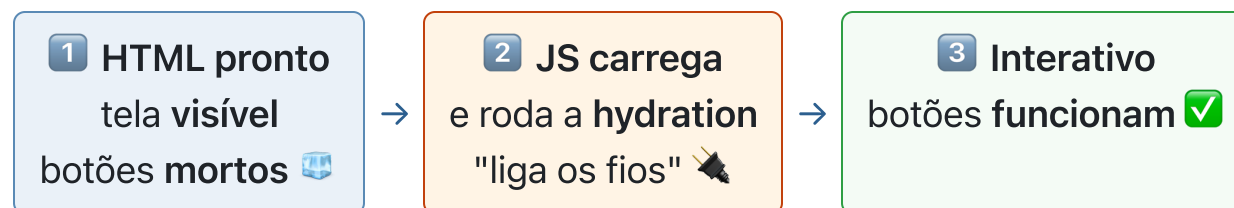
Nesta aula usamos dois sentidos:

- 1. **App pronto** — o JS terminou de ligar a tela (próximos slides: por que o teste quebra)
- 2. **Reidratar estado offline** — o app reabre sem internet e **recupera** os dados salvos (banco offline)

| Mesma ideia nos dois: **o app só é útil depois de "acordar"** — e o teste precisa **esperar** isso.

Hydration — o que é e por que quebra teste

Em apps com **SSR** (o servidor já manda o HTML pronto, ex.: Next.js) a página aparece **antes** de o JavaScript chegar:



Hydration = o JS "acordar" o HTML que já está na tela (anexar os cliques). Se o teste clica fazendo **1**, o clique é **perdido** — e o teste falha às vezes.

O **CineFav** é uma SPA **sem SSR** (React + Vite): o HTML vem vazio e o React monta tudo no navegador. Não tem hydration, mas tem o **mesmo risco**: clicar antes de o app estar **pronto**.

O fluxo do teste — por que quebra (e por que só às vezes)

❌ Sem esperar o app ficar pronto

- 1 `page.goto('/')`
- 2 HTML na tela — **botões ainda mortos** 📄
- 3 `click()` ← o clique cai no vazio 🖱️
- 4 `expect(...).toBeVisible()` → **falha**

✅ Esperando o sinal de pronto

- 1 `page.goto('/')`
- 2 `espera data-app-ready="true"` ⌚
- 3 `click()` ← agora o botão responde ✅
- 4 `expect(...).toBeVisible()` → **passa**

Por que falha às vezes: é uma **corrida** entre o teste e o JavaScript.

Máquina	Quem chega primeiro	Resultado
Seu notebook (rápido)	o JS liga antes do clique	✅ passa
CI / máquina lenta	o clique chega antes do JS	❌ falha

É o clássico "**passa na minha máquina e falha no CI**" (teste *flaky*). A cura não é `waitForTimeout(2000)` : é **esperar um sinal** de que o app está pronto.

Testando: esperar o app ficar pronto

```
// CineFav: o app marca <html data-app-ready="true"> quando o React terminou de montar
await page.goto('/qa');
await expect(page.locator('html')).toHaveAttribute('data-app-ready', 'true');

await page.getByTestId('movielist-search-button').click(); // agora o clique é seguro
await expect(page.getByTestId('search-screen')).toBeVisible();
```

- **Esperar um sinal de pronto** (atributo, elemento, texto), não um tempo fixo (`waitForTimeout`)
- Em apps **SSR** (Next.js) o princípio é o mesmo: espere algo que **só existe depois de hidratar**
- Evite `networkidle` como regra: é um palpite ("a rede sossegou"), não prova que o app está interativo
- Cada ação do Playwright (`click` , `fill`) já **espera o elemento ficar clicável** — mas não sabe se o JS ligou

É o **spec 04** do lab (teste 1): `data-app-ready` .

O que é um Service Worker (SW)?

Um **Service Worker** é um **script** que o navegador roda em **segundo plano**, separado da página. Pense num **porteiro** entre o seu app e a internet: toda requisição passa por ele.

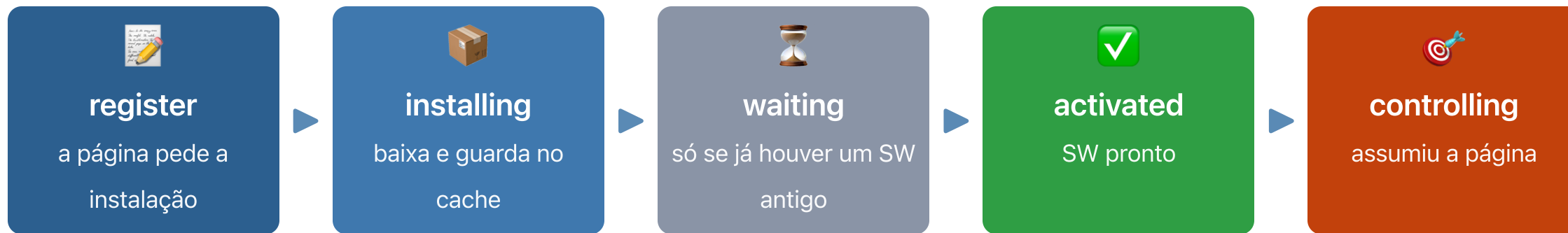


- **Intercepta** cada `fetch` do app e escolhe: responder do **cache** ou ir na **rede**
- É por isso que a PWA **abre sem internet**: o porteiro entrega da prateleira (cache)
- Não tem acesso à tela (não mexe no DOM) · só existe em **HTTPS** (ou `localhost`)

Sem SW não há offline — e é isso que o Playwright precisa conferir.

Ciclo de vida do SW — em uma imagem

Primeira visita (o SW nasce e instala):



⚠ **janela de perigo** — ativo, mas **ainda não controla** a página

✍ Onde o teste espera

1. `active.state === 'activated'`
2. `controller !== null` ← só então derrubar a rede

🔄 Versão nova (atualização)

installing → **waiting** (espera o usuário clicar "**Atualizar**") →
activated → `controllerchange`

Teste offline que **pula a etapa** 🎯 cai exatamente na janela de perigo — e falha às vezes.

Ciclo de vida do SW — e por que isso complica o teste

Fase	O que acontece	No teste
register	a página pede ao navegador pra instalar o SW	só no build (<code>npm run preview</code>) — não no <code>npm run dev</code>
installing	baixa os arquivos e guarda no cache	esperar terminar
waiting	versão nova espera a antiga fechar	é o toast "Nova versão — Atualizar"
activated	SW pronto	<code>reg.active.state === 'activated'</code>
controlling	o SW assumiu a página	<code>navigator.serviceWorker.controller !== null</code>

 **activated** ≠ **controlando**. Na 1ª visita o SW ativa, mas só passa a **controlar** a página depois. Teste offline sem esperar o `controller` falha de forma intermitente.

Testando o SW com Playwright — o essencial

```
test('SW ativo e controlando a página', async ({ page }) => {  
  await page.goto('/qa'); // 1. visita online → o SW registra e instala  
  
  await expect  
    .poll(() => // 2. espera o SW ficar "activated"  
      page.evaluate(async () => {  
        const reg = await navigator.serviceWorker.getRegistration();  
        return reg?.active?.state;  
      })),  
    .toBe('activated');  
  
  await page.waitForFunction( // 3. ...e CONTROLAR esta página  
    () => navigator.serviceWorker.controller !== null,  
  );  
});
```

Roda contra o `build ( vite preview ).O playwright.config.ts` do lab já faz isso.

Estratégias de cache — qual usar e como testar

Estratégia	Ideia	No CineFav	Como testar
CacheFirst	cache primeiro; rede só se faltar	posters do TMDb (não mudam)	<code>caches.open('cinefav-posters')</code> tem entradas
NetworkFirst	rede primeiro; cache se cair	<code>/api/movies.json</code> (dados mudam)	<code>setOffline(true) + reload</code> → ainda lista os filmes
StaleWhileRevalidate	mostra o cache já e atualiza ao fundo	(não usado)	conferir o dado velho na hora e o novo depois

O app shell (HTML, JS, CSS) vai no *precache*: o SW guarda tudo na instalação. É o que faz o app abrir offline.

Atualização do SW — o toast "Nova versão"

Quando você publica uma versão nova, o SW novo **espera (waiting)** até a antiga fechar. A PWA do CineFav **avisa o usuário**, e ele escolhe quando atualizar:

1. SW novo instala e fica **waiting**
2. app mostra o toast " ✨ **Nova versão — Atualizar**"
3. clique → o SW novo assume (`controllerchange`) e a página recarrega

Testar isso de ponta a ponta pede **dois builds** (versão A e B). No lab conferimos o essencial: o SW fica **ativo e controlando**. Os cenários de *waiting* são onde mora o bug.

PWA — manifest validation

```
test('manifest is valid', async ({ page }) => {  
  const response = await page.goto('/manifest.json');  
  const manifest = await response?.json();  
  
  expect(manifest.name).toBeTruthy();  
  expect(manifest.short_name).toBeTruthy();  
  expect(manifest.start_url).toBe('/');  
  expect(manifest.display).toBe('standalone');  
  expect(manifest.icons.length).toBeGreaterThan(0);  
  expect(manifest.icons.find((i) => i.sizes === '192x192')).toBeDefined();  
  expect(manifest.icons.find((i) => i.sizes === '512x512')).toBeDefined();  
});
```

PWA — instalação: o que o teste confere

Como funciona: o Chrome vê que o app é **instalável** (manifest + Service Worker + HTTPS) → dispara o evento `beforeinstallprompt` → o app **guarda** o evento e liga o botão "Instalar" (no CineFav: tela `/pwa`).

O problema no teste: o navegador automatizado **nunca** dispara esse evento (botão fica desligado) e o diálogo do sistema **não dá pra automatizar**. **A solução:** **simular o evento** e testar o **nosso código**:

```
await page.goto('/pwa');
await expect(page.getByTestId('pwa-install-button')).toBeDisabled(); // sem evento = sem instalar

await page.evaluate(() => { // simula o que o Chrome faria
  const e: any = new Event('beforeinstallprompt');
  e.prompt = async () => {};
  e.userChoice = Promise.resolve({ outcome: 'accepted' });
  window.dispatchEvent(e);
});

await expect(page.getByTestId('pwa-install-button')).toBeEnabled(); // botão liga!
```

Teste 5 do bônus `09` : prova que o **app reage** ao evento — não que o Chrome instala.

PWA — offline mode test

```
test('catálogo continua acessível offline', async ({ page, context }) => {  
  await page.goto('/qa');  
  await expect(page.getByTestId('movielist-grid')).toBeVisible();  
  
  // SW "activated" NÃO basta: precisa CONTROLAR a página  
  await page.waitForFunction(() => navigator.serviceWorker.controller !== null);  
  
  await context.setOffline(true); // perda total de rede  
  await page.reload();  
  
  await expect(page.getByTestId('movielist-grid')).toBeVisible();  
  await expect(page.getByTestId('offline-banner')).toBeVisible();  
});
```

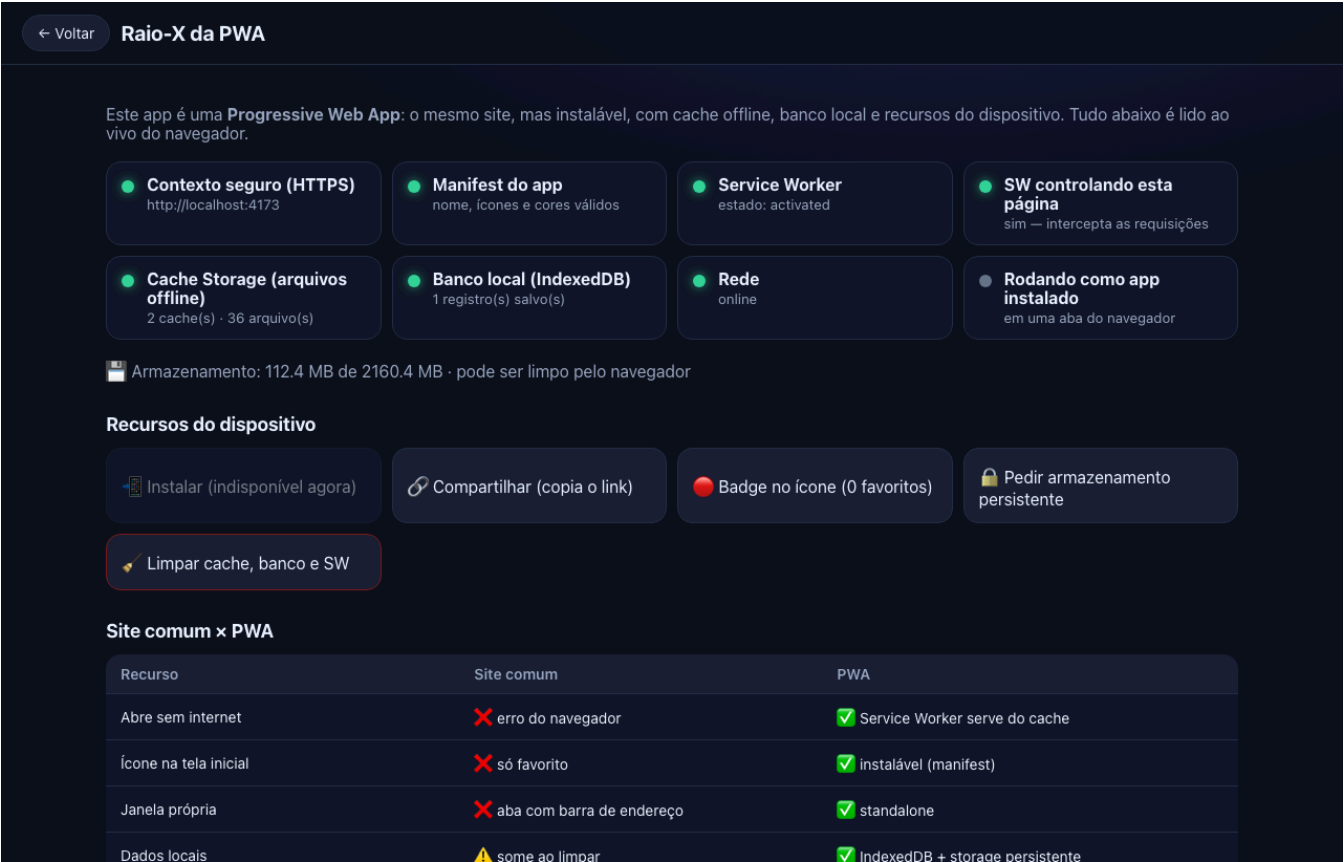
Sequência: visita online → SW controla → offline → reload. Pular o 2º passo é o erro mais comum.

Site comum x PWA — o que muda de verdade

Situação	 Site comum	 PWA
Sem internet	 "Sem conexão" (o dinossauro)	 abre normalmente (Service Worker)
Atalho no celular	 só um favorito do navegador	 ícone próprio na tela inicial
Como abre	 numa aba, com barra de endereço	 janela própria, sem barra (parece app)
Versão nova	 aparece sozinha, a cada visita	 avisa: "Nova versão — Atualizar"
Dados do usuário	 somem fácil	 ficam no aparelho (IndexedDB)
Recursos do celular	 quase nenhum	 compartilhar, badge no ícone, atalhos

No CineFav: abra `/pwa` ("Raio-X") — mostra **ao vivo** o que o navegador sabe: HTTPS, manifest, Service Worker, cache, banco e rede.

Raio-X da PWA — o app se auto-diagnostica



Roda no **build** (`npm run build && npm run preview`): em `npm run dev` não há Service Worker, então o painel mostra tudo apagado — bom pra **comparar**.

Onde o app guarda coisas offline — 2 "gavetas"

Cache Storage

*a gaveta do **Service Worker***

Guarda: arquivos prontos

No CineFav: o app em si (HTML, JS, CSS), a lista `movies.json` e os **posters**

Quem usa: o SW, sozinho

Ver no navegador: DevTools → Application → **Cache storage**

IndexedDB

*a gaveta do **código do app***

Guarda: dados que o app lê e escreve

No CineFav: as respostas da API de filmes (pra lista abrir offline) e **seus comentários**

Quem usa: o código do app

Ver no navegador: DevTools → Application → **IndexedDB** → **cinefav**

Como o Playwright olha dentro de cada uma:

```
cache.open('cinefav-posters') · indexedDB.open('cinefav') — via page.evaluate()
```

Regra prática: **arquivo pronto** vai no *Cache Storage*; **dado que o app muda** vai no *IndexedDB*.

Features de dispositivo — e como testar cada uma

Feature	API	Teste Playwright
Instalar	<code>beforeinstallprompt</code>	<code>dispatchEvent</code> do evento falso → botão habilita
Atalhos do ícone	<code>manifest.shortcuts</code>	<code>fetch(manifest)</code> e conferir as URLs
Atualização	<code>registerType: 'prompt'</code>	toast "Nova versão" + botão Atualizar
Badge no ícone	Badging API	feature-detect (só app instalado)
Compartilhar	Web Share	fallback copia o link
Offline real	<code>context.setOffline(true)</code>	reload + <code>offline-banner</code>

Bônus (não pontua): `tests/e2e-bonus/09-pwa-painel` e `10-banco-offline` — rode `npm run test:bonus`.

Fila offline — comentar sem internet

No CineFav, comentar **sem rede** não perde nada:

1. Sem internet → o comentário vai pra uma **fila** local (🕒 "na fila")
2. A internet volta → o app **envia** e marca ✓ "enviado"

```
await page.goto('/movie/603');
await context.setOffline(true); // 1. derruba a rede

await page.getByTestId('comment-input').fill('Clássico!');
await page.getByTestId('comment-submit-button').click(); // 2. comenta offline
await expect(statusDoComentario).toHaveAttribute('data-status', 'pending'); // 🕒 na fila

await context.setOffline(false); // 3. a rede volta
await expect(statusDoComentario).toHaveAttribute('data-status', 'synced'); // ✓ enviado
```

É o teste 3 do bônus `10-banco-offline`. O **Background Sync** (o SW enviando com o app fechado) existe, mas só no Chrome; o CineFav usa o evento `online`, que funciona em todo navegador.

Missão PWA — 15 min (sozinho ou em dupla)

Abram <https://cinefav-web.vercel.app> (no **celular** também!) e sintam a diferença entre **site comum** e **PWA**:

1. **Criar conta** na tela de login e entrar
2. **Instalar** o app (Chrome: ícone ⊕ na barra de endereço · celular: "Adicionar à tela inicial") e **abrir pelo ícone novo** — tem barra de endereço?
3. Abrir `/pwa` (Raio-X): quais linhas estão verdes? alguma cinza?
4. Ligar o **modo avião** (ou DevTools → Network → **Offline**) e abrir o app de novo — funciona? Compare com um site comum (ex.: g1.com) offline
5. Abrir um filme, **comentar sem rede** (🕒), religar a rede (✓)
6. DevTools → **Application**: achar o **Service Worker**, o **Cache Storage** e o **IndexedDB**

! Não vale nota. Anote **3 diferenças** que você percebeu e mande no chat.

 Missão PWA — agora pense como testador

Cada passo da missão vira um **teste automatizado**. Qual `expect` provaria isso?

O que você fez	Teste que prova	Onde está
App instalável	manifest válido + botão "Instalar" liga	spec 05 (#2) · bônus 09 (#5)
Abre sem internet	<code>setOffline(true)</code> + <code>reload</code> + <code>offline-banner</code>	spec 05 (#3)
Raio-X verde (SW, cache, HTTPS)	<code>data-ok="true"</code> em cada linha de <code>/pwa</code>	bônus 09 (#2)
Comentou offline e sincronizou	<code>data-status</code> <code>pending</code> → <code>synced</code>	bônus 10 (#3)
Dado salvo no aparelho	ler o <code>IndexedDB</code> com <code>page.evaluate()</code>	bônus 10 (#2)

Pergunta pra turma: qual desses passos **só um humano** consegue verificar? *(dica: a janela "sem barra de endereço" e o diálogo de instalação do sistema)*

Lighthouse CI — visão geral

O **Lighthouse** mede performance, acessibilidade e boas práticas. O **Lighthouse CI** roda isso **várias vezes** e **reprova** se alguma meta (*budget*) for ultrapassada. **Já vem no projeto** — não precisa instalar nada:

```
cd exercicios/01-lab-web-pwa-playwright/pratica
npm run build          # o Lighthouse mede o BUILD (como em produção)
npm run lighthouse     # roda o `lhci autorun` do projeto (3 execuções)
```

```
// lighthouseci.json (já está no projeto)
{
  "ci": {
    "collect": {
      "staticDistDir": "./dist",
      "isSinglePageApplication": true,
      "url": ["http://localhost/login"],
      "numberOfRuns": 3
    },
    "assert": { "assertions": { "categories:performance": ["error", { "minScore": 0.9 }], "...": "..."} }
  }
}
```

`staticDistDir`: o Lighthouse CI **sobe o dist/ sozinho**. São **3 execuções** porque uma só varia demais.

Performance budgets

Conceito de **Tim Kadlec (2013)** popularizado por web.dev. No lab, os budgets estão no `lighthouse.json`:

```
"assertions": {
  "categories:performance": ["error", { "minScore": 0.9 }],
  "categories:accessibility": ["error", { "minScore": 0.9 }],
  "categories:best-practices": ["error", { "minScore": 0.9 }],
  "first-contentful-paint": ["error", { "maxNumericValue": 2000 }],
  "largest-contentful-paint": ["error", { "maxNumericValue": 2500 }],
  "total-blocking-time": ["error", { "maxNumericValue": 300 }]
}
```

💡 **Budget de tamanho** (opcional): `"resource-summary:script:size": ["error", { "maxNumericValue": 300000 }]` reprova o PR se o JavaScript passar de ~300 KB. **Por quê?** App engorda aos poucos (+20 KB aqui, +30 KB ali) e ninguém percebe até ficar lento — é a *"morte por mil cortes"*. O budget barra o **PR que estourou**, não só o último.

Lighthouse CI no GitHub Actions

```
name: Lighthouse CI
on: [pull_request]

jobs:
  lighthouse:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: npm ci
      - run: npm run build
      - run: npm run lighthouse      # já vem no projeto (npx lhci autorun)
    env:
      LHCI_GITHUB_APP_TOKEN: ${ secrets.LHCI_GITHUB_APP_TOKEN }
```

Resultado posta comment no PR com link pra report. Falha bloqueia merge.

Cobertura de código — o que o relatório mede

Cobertura = **quanto do código-fonte foi executado** pelos testes. Quatro métricas:

Métrica	Pergunta que responde
Statements	quantas instruções rodaram?
Branches	os dois lados de cada <code>if / ?:</code> rodaram?
Functions	cada função foi chamada ao menos 1 vez?
Lines	quantas linhas rodaram?

No e2e do CineFav: cobertura **V8 do Chromium** (`page.coverage.startJSCoverage()`) + sourcemap → relatório por arquivo de `src/`.

Relatório real — suíte e2e do CineFav

All files

78.47% Statements

113/144

67.39% Branches

62/92

58.82% Functions

60/102

77.85% Lines

109/140

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements	Branches	Functions	Lines
App.tsx	100%	7/7	100%	0/0
components	75%	15/20	58.33%	7/12
main.tsx	100%	1/1	100%	0/0
screens	84%	42/50	74.46%	35/47
services	72.72%	24/33	47.36%	9/19
store	71.87%	23/32	78.57%	11/14
utils	100%	1/1	100%	0/0

Code coverage generated by Istanbul at 2026-10-03T19:01:00.670Z

78% statements · 67% branches · 59% functions. No vermelho: services (branches 47%) e utils (functions 44%).

Dentro de um arquivo — o que ficou de fora

All files / store favorites.ts

57.89% Statements 11/19 62.5% Branches 5/8 55.55% Functions 5/9 55.55% Lines 10/18

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

```
1 // src/store/favorites.ts
2 //
3 // Favoritos persistidos em localStorage, com useSyncExternalStore pra
4 // qualquer tela reagir a mudanças (mesma semântica do favoritesStore mobile).
5
6 import { useSyncExternalStore } from 'react';
7
8 26x const KEY = 'cinefav-favorites';
9
10 type Listener = () => void;
11 const listeners = new Set<Listener>();
12
13 13x function read(): number[] {
14 13x   const raw = localStorage.getItem(KEY);
15 13x   if (!raw) return [];
16   try {
17     return JSON.parse(raw) as number[];
18   } catch {
19     return [];
20   }
21 }
22
23 26x let snapshot: number[] = [];
24 let snapshotRaw = '';
25
26 277x function getSnapshot(): number[] {
27 277x   const raw = localStorage.getItem(KEY) ?? '[]';
28 277x   if (raw !== snapshotRaw) {
29     snapshotRaw = raw;
30     snapshot = read();
31   }
32   return snapshot;
33 }
34
```

Verde = executado (13x , 277x). Vermelho = nunca executado: o `try/catch` do `JSON.parse` em `store/favorites.ts` .

Cobertura ≠ qualidade

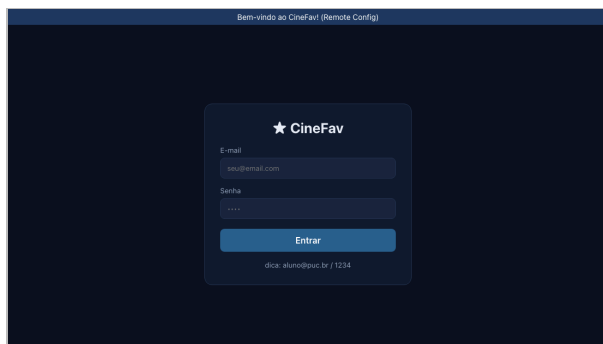
- Linha executada **não** é comportamento verificado: um teste **sem** `await` / `matcher` executa o código e dá 100% de cobertura sem checar nada
- Use o relatório pra achar o que **não** foi testado (o vermelho), não como meta de nota
- Gate sensato no CI: piso por **branches** (ex.: 70%), não 100%

Na Aula 6: por que **mutation testing** prova o que a cobertura só sugere.

Demo ao vivo — Firebase Remote Config em todos os PWAs

Mudar o app **sem novo deploy**: o banner do topo vem do parâmetro `banner_message` do **Firebase**.

1. **Todo mundo** abre <https://cinefav-web.vercel.app> e entra (ou cria conta)
2. O banner aparece no topo: *"Bem-vindo ao CineFav! (Remote Config)"*
3. O professor **edita o texto no console do Firebase** e clica **Publicar**
4. Em até **15 segundos** o banner muda **na tela de todo mundo** — sem F5, sem deploy



É o princípio de *feature flag*: ligar/desligar coisas em produção sem publicar versão nova.

Testando API de terceiro — sem bater no Firebase

Em teste, **não** chamamos o Firebase de verdade (rede + credencial = flaky). Interceptamos:

```
await page.route('**/firebaseremoteconfig.googleapis.com/**', (route) =>
  route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify({ entries: { banner_message: 'Promo de teste ativa!' } }),
  }),
);
await page.goto('/qa');
await expect(page.getByTestId('remote-banner')).toHaveText('Promo de teste ativa!');
```

Bônus (não pontua): `tests/e2e-bonus/08-remote-config-banner.spec.ts` — `npm run test:bonus`. Mesmo `page.route()` da Aula 2.

Intervalo

15 minutos

*Na volta: vamos pro **lab** — specs 03 (visual), 04 (SPA), 05 (PWA/offline) + Lighthouse CI.*

Aproveite pra abrir o terminal em `exercicios/01-lab-web-pwa-playwright/pratica` e rodar `npm install` e `npx playwright install chromium`.

Lab — specs 03, 04 e 05 + Lighthouse (90min)

```
cd exercicios/01-lab-web-pwa-playwright/pratica · ls tests/e2e
```

1. **Spec 03 — visual regression** (25min): `toHaveScreenshot` , 3 viewports, máscara. Teoria nos slides da Aula 2
2. **Spec 04 — SPA** (20min): `data-app-ready` , navegação client-side, lazy chunk
3. **Spec 05 — PWA/offline** (30min): SW ativo, manifest, offline com controller
4. **Lighthouse CI** (15min): `lighthouseci.json` + 3 budgets rodando

Tudo em `/qa` (ambiente determinístico), contra o **build** (`vite preview`) — SW não existe no dev server.

Pitfalls comuns

- ✗ Testar SPA sem aguardar hydration — falha intermitente
- ✗ Reset de cache Service Worker entre tests — esquecer = test pollution
- ✗ Lighthouse CI rodando uma vez só — variação alta, configure 3+ runs
- ✗ Budget muito apertado — bloqueia PRs legítimos por 10ms
- ✗ Não monitorar Web Vitals em produção — divergência entre lab e prod
- ✗ Offline sem esperar o SW controlar a página — `activated` não basta, falta `controller`
- ✗ `expect` de locator sem `await` — verde sem checar, ou erro "page closed"

Atividade — Lab Web + PWA (15 pts)

Entrega: fork + PR. Prazo: ver Canvas. Enunciado completo no repo.

#	Critério	Pontos
1	Auth state reuse (<code>storageState</code>)	2
2	Network mocking (<code>route()</code>)	2
3	Visual regression, 3 viewports	3
4	SW lifecycle testado	2
5	Offline mode (<code>setOffline</code>)	3
6	Lighthouse CI, 3 budgets	3

Eliminatório: suíte 100% verde em 3 runs consecutivos.

Leituras obrigatórias

- Archibald, J. — *Service Workers Testing Strategies* (web.dev)
- web.dev — *Lighthouse CI* (oficial)
- Lock, A. — *Testing Service Workers* (artigo)
- **Mobile.dev Docs** — preparação Aula 4

Próxima aula — Aula 4

Automação Mobile com Maestro

- YAML declarativo
- Conditional flows + runFlow
- Maestro Studio
- Comparativo Maestro vs Appium vs Detox
- Cloud devices

Bons estudos!

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith